

Data Platform System Design

1. Streaming Clickstream

Path: `FastAPI_A` → `Kafka2` → `ClickHouse.raw` (Kafka Engine) → `Grafana`

Key Features:

- **Zero-copy, high-throughput ingestion** of Avro-encoded clickstream events.
- **Reliability & Operational Safety:**
 - Asynchronous DLQ pattern → malformed events redirected to `clickhouse.dead_letter_events`.
 - Strict consumer-group isolation → ClickHouse Kafka Engine operates independently from NiFi & archival pipelines.
- **Backpressure & Lag Handling:**
 - Kafka as primary buffer. Lag builds if ClickHouse slows or goes down.
 - Lag drain throttling using `kafka_max_block_size` and `kafka_poll_max_batch_size` to prevent insert storms during recovery.
 - Small buffering windows in NiFi absorb minor out-of-order events.
- **Late/Out-of-Order Events:**
 - Partitioning by `event_ts`.
 - Deduplication via `ReplacingMergeTree(FINAL)` ensures late-arriving or duplicate events are upserted safely.
 - Materialized views or scheduled merges used to avoid heavy `SELECT ... FINAL` on hot query paths.
- **Schema Governance:**

- Streaming merges enforce schema compatibility via MergeTree constraints.
- Schema evolution versioned via Schema Registry.

2. Streaming Operational Events

Path: FastAPI_B → Microservices (PostgreSQL + Redis) → Kafka1 → NiFi1 → ClickHouse.enriched → Grafana

Key Features:

- NiFi pauses consumption if ClickHouse slows → prevents cascading failures.
- Deduplication via `ReplacingMergeTree(event_id)` ensures effectively-once semantics.
- Redis provides hot reference data & rate-limit counters for APIs.
- Late-event handling implicitly covered by MergeTree constraints.

3. Batch / master_events Layer

Landing / Bronze: FastAPI_C → Postgres → DuckDB (Parquet conversion + file_id validation) → S3 Bronze

- Batch CSV upload, temporary storage, metadata persisted in Postgres.
- Ingestion independent of ClickHouse success.

Processing / Curated: S3 Bronze → Airflow → Spark Transform → ClickHouse.master_events → Grafana

- Spark performs heavy joins, denormalization, aggregation.
- **Schema Enforcement:**
 - Validates batch data against formal `master_events` schema contract in Postgres.
 - Streaming merges enforce compatibility via ClickHouse MergeTree constraints.

- **Insert Performance:**
 - Spark batches writes in optimal insert sizes (10k–100k rows per insert) using native ClickHouse connectors.
- **Watermarks:** Stored in Postgres, used by Airflow for replay-safe incremental processing.
- **Full Rebuild Capability:** Batch paths can fully rebuild `master_events` from S3 Bronze.

4. Archival Layer

`Kafka1/Kafka2 → NiFi2 → S3`

- Immutable archive of all events.
- Independent consumer groups → archival ingestion does not affect streaming/enriched paths.
- Supports disaster recovery & full replay.

5. Ingestion Design

Entry Points:

- `FastAPI_A` → clickstream
- `FastAPI_B` → internal events
- `FastAPI_C` → batch CSV uploads

FastAPI Responsibilities:

- Schema validation (Avro + Schema Registry for streaming, CSV validation for batch)
- Generate deterministic UUIDv7 `event_id`

- Persist operational state to PostgreSQL
- Read/write hot reference data & rate-limit counters via Redis (HA)
- Publish events to Kafka (idempotent producers, acks=all)
- Attach trace_id for observability

6. Kafka Design

Cluster: 3–5 brokers, Multi-AZ, Replication Factor=3, idempotent producers, acks=all, min.insync.replicas=2

Topics & Partitioning:

- `clickstream.events` → partition by `user_id`
- `internal.events` → partition by `aggregate_id`
- `dlq.*` → dead-letter queue
- Independent consumer groups for ClickHouse vs NiFi/archival

Retention:

- Clickstream: 3–5 days
- Internal: 7–14 days
- DLQ: 15–30 days
- Reference topic: log-compacted

Backpressure & Replay:

- Kafka primary buffer; NiFi pauses on slow ClickHouse
- Replay via Kafka retention
- Full rebuild from S3 Bronze

Throttling Knobs:

- `kafka_max_block_size` & `kafka_poll_max_batch_size` control Kafka Engine drain speed after downtime.

7. Storage Design (S3 / MinIO)

S3 Raw Layer:

- Immutable, schema-versioned Avro
- Partitioned by `yyyy/mm/dd/hour`
- System of record, replayable, not analytics-optimized

ClickHouse.master_events (Curated Layer):

- Merges streaming + batch, deduplicated, denormalized
- Business logic applied
- Partitioned by `toYYYYMM(event_ts)`
- Fully rebuildable from S3 raw

8. Processing Design

Streaming Path:

- `FastAPI_A/B` → `Kafka` → `ClickHouse (raw/enriched)` → `Grafana`
- At-least-once delivery, ordering within partitions
- Deduplication via `ReplacingMergeTree(FINAL)` and idempotent producers

Batch Path:

- `FastAPI_C` → `DuckDB` → `S3 Bronze` → `Airflow` → `Spark` → `ClickHouse.master_events`

- Schema enforcement, heavy joins, denormalization, aggregation
- Watermarks for replay-safe incremental processing

Data Quality Observability:

- Assertions: row counts, null rates, referential integrity, value distributions, schema validation
- Failure Handling: DAG halts, alerts fire, partitions quarantined
- Reconciliation DAG: scheduled cross-checks ClickHouse vs S3 Bronze, triggers alerts/backfill if discrepancies

9. ClickHouse Serving Model

- Sharded + replicated, distributed tables, AZ replication
- Tables:
 - `Raw_realtime`: MergeTree, partition `toYYYYMM(event_ts)`, order `(event_ts, user_id)`
 - `Enriched`: ReplacingMergeTree(event_id) using FINAL
 - `Master_events`: pre-joined, merges batch + streaming
- TTL policies: Raw 30–90 days, aggregates retained longer
- Fully rebuildable from S3

10. PostgreSQL (Operational DB)

- Stores: API state, upload metadata, job status, reference data, auth, checkpoints, watermarks
- HA: primary + replicas, WAL replication, automated failover

11. Orchestration (Airflow)

- DAGs: Bronze → Spark → Master Events, ClickHouse refresh, DQ, reconciliation, backfill
- Scheduling: Hourly (near-real-time batch), Daily (finance aggregates)
- Backfill: parameterized DAGs using watermark logic, reads from Bronze

12. Observability

- Metrics: Kafka lag, under-replicated partitions, leader elections; NiFi queue size/backpressure; ClickHouse insert/query latencies; S3 growth/completeness; Airflow DAG/task metrics
- Logs: centralized (ELK/Loki)
- Tracing: FastAPI → Kafka → ClickHouse trace_id propagation
- Alerts: Grafana for SLA breaches, DQ failures

13. Security & Governance

- IAM separation (analytics vs ops)
- Schema Registry enforcement, data contracts
- DQ checks in Spark + Airflow, DLQ for malformed events
- Data lineage tracked via Airflow metadata

14. Disaster Recovery & Reprocessing

- Multi-AZ Kafka, Multi-region S3 replication, ClickHouse replication, Postgres failover, backups
- Short-term replay: Kafka retention
- Long-term rebuild: S3 Bronze

15. Horizontal Scale Points

- FastAPI replicas (load balanced)
- PostgreSQL primary + read replicas, PgBouncer
- Redis Cluster + Sentinel
- Kafka brokers + topic partitions
- NiFi cluster nodes
- Spark executors
- Airflow workers
- ClickHouse shards
- Grafana instances